
EPISTEMIC CAPTURE AND THE ACTION BOUNDARY: CORRIGIBILITY FOR LEARNED AND AGENTIC SYSTEMS

A PREPRINT

Anivar A Aravind 
Independent Researcher
Bengaluru, India
ping@anivar.net

April 2026

Abstract

This paper extends the corrigibility framework to learned and agentic systems, where deterministic rule execution is replaced by probabilistic inference. Three structural pressures constrain corrigibility in learned systems: opacity of inference (requiring LWD-R transparency), concentration of training resources (compute capture), and acceleration of automated action (governance denial of service). The action boundary protocol separates probabilistic inference from deterministic execution. The invariant holds: systems remain corrigible if and only if the five conditions close the loop under stochastic verification.

Dependency. This paper presupposes the companion paper [Aravind, 2026], which derives the five corrigibility tests (EXIT, CODE, AUDIT, GOVERN, FORK) from control theory and commons governance. The present paper does not re-derive these tests; it extends their verification methods to stochastic systems.

Keywords Epistemic Public Infrastructure · AI Governance · Corrigibility · Action Boundary Protocol · Compute Capture · LWD-R

How to Read This Paper. Corrigibility is the minimal architectural condition that keeps public systems reversible. The argument has three parts: learned systems require bounded error propagation (the conceptual invariant); five control-layer conditions extend to stochastic inference (the architectural constraints); and these constraints discriminate between AI-mediated systems (the applied evaluation). Case studies illustrate discriminative power, not exhaustive institutional evaluation.

Executive Summary

When infrastructure incorporates machine learning, it no longer merely processes transactions. It generates claims about reality. This creates **epistemic public infrastructure (EPI)**.

Traditional transparency is insufficient. Reconstruction of model behavior requires disclosure of logic, weights, data provenance, and representational schema (**LWD-R**). Practical forkability is constrained by compute asymmetry. Performance degrades under distributional shift.

The **action boundary protocol** separates probabilistic inference from deterministic execution through enforceable validation layers. Without such boundaries, agentic systems exercise unbounded epistemic power.

1 Introduction

The companion paper [Aravind, 2026] establishes a corrigibility framework for Digital Public Infrastructure (DPI), deriving five structural tests from cybernetics, commons governance, and free software. That framework

addresses **deterministic** infrastructure: ledgers, registries, and payment rails where source code is the arbiter of behavior. This paper extends the framework to **stochastic** infrastructure, what we term **Epistemic Public Infrastructure (EPI)**: systems where state decisions emerge from learned parameters rather than explicit logic.

Definition 1.1 (Epistemic Public Infrastructure). EPI comprises AI systems that determine citizen outcomes at population scale, including: identity verification models, welfare eligibility systems, public-facing chatbots with administrative authority, and autonomous agents operating within mandatory infrastructure.

1.1 DPI and EPI: Structural Contrast

Digital Public Infrastructure (DPI) processes transactions through deterministic logic. Epistemic Public Infrastructure (EPI) produces classifications, predictions, or inferences through learned parameters.

Table 1: Structural contrast between DPI and EPI

DPI	EPI
Deterministic rule execution	Stochastic learned inference
Logic transparency sufficient	Weights, data, and representations matter
Forkability tied to software	Forkability tied to compute capacity
Static certification possible	Performance degrades under distribution shift

Corrigibility in EPI therefore requires additional structural conditions beyond those sufficient for deterministic systems.

Tiered Adoption. EPI systems vary in consequence severity. The framework defines three deployment tiers with increasing compliance requirements:

Table 2: EPI deployment tier classification

Tier	Use Cases	Requirements
trivial	Translation, summarization	Action boundaries required
decision_support	RAG over government data	Open weights, local data storage
high_stakes	Welfare, policing, judicial	Full LWD-R, human-in-loop

The tier classification is a governance decision, not a technical one. Systems must declare their tier; auditors verify appropriateness.

Why “Epistemic”? The term is structurally precise. In deterministic DPI, infrastructure manages *what happened*: the **Ledger State** (a payment was made, an identity was verified). In learned systems, infrastructure manages *what is true*: the **Inference State** (a person is “eligible,” a transaction is “fraudulent,” a citizen is a “risk”). This shift from transactional facts to administrative knowledge constitutes a transfer of **epistemic sovereignty**.

When a state uses a model to determine eligibility, it is no longer merely processing data; it is producing a *claim to knowledge*. If that claim is hidden in a latent space, the infrastructure has captured the state’s ability to know or contest the truth. The “R” in the LWD-R Triad (Representation, Section 3) is the epistemic anchor: if the categories through which the state “sees” the world are proprietary and non-contestable, the state has delegated its interpretive sovereignty. Citizens are not merely locked out of a database; they are locked into a reality they cannot define.

By naming this domain **Epistemic Public Infrastructure**, the framework asserts that the “truth” produced by an AI system must satisfy the same corrigibility tests as a legal fact. The question shifts from “Does the AI work?” to “Is the knowledge produced by this system structurally falsifiable and reversible?”

Architectural Closure. DPI solves the problem of **transactions** (Scale and Velocity). EPI solves the problem of **meaning** (Stochasticity and Representation). Once the infrastructure for creating administrative reality (EPI) is bounded by the same five structural tests as the infrastructure for processing payments (DPI), the framework achieves closure. The theory has scaled from the registry to the world model.

The five structural tests are:

- **EXIT**: Reversibility of participation
- **CODE**: Inspectability of logic
- **AUDIT**: Independent verification
- **GOVERN**: Binding constraint
- **FORK**: Capacity to reproduce

The Cybernetic Foundation. These tests are not arbitrary normative preferences but derive from control theory. Ashby’s Law of Requisite Variety [Ashby, 1956] establishes that a controller must possess at least as many response states as the system it governs. Applied to infrastructure: governance mechanisms must match the variety of harms a system can produce. The five tests operationalize this requirement as a closed feedback loop: EXIT provides the error signal (affected parties can refuse), AUDIT acts as the sensor (errors are detectable), CODE ensures transmission clarity (logic is inspectable), GOVERN serves as the actuator (correction is binding), and FORK provides selection pressure over the entire loop (the system can be reproduced if operators refuse correction).

The Open-Loop Instability Theorem. When any test fails, the feedback loop opens. Open-loop systems inevitably drift toward instability because errors accumulate without correction. The companion paper proves formally that partial compliance (meeting some tests while failing others) is functionally equivalent to complete failure. A system with EXIT but no GOVERN allows users to detect harm but not correct it; a system with CODE but no FORK allows inspection but not competition. The five tests form an *irreducible* set: removing any one collapses the corrective apparatus.

This paper stress-tests the framework against learned systems: infrastructure where behavior is determined by parameters learned from data (AI/ML) rather than explicit code logic. If the five tests are truly fundamental, they must apply not only to deterministic infrastructure but also to stochastic systems where traditional verification assumptions collapse.

The Challenge. Learned systems dismantle the assumption that source code is the sole arbiter of behavior. In AI, identical inputs may yield stochastic outputs; behavior is defined by training data; and opacity is often inherent rather than contrived. Yet the collapse of deterministic transparency does not invalidate corrigibility’s structural necessity. As systems become less intelligible, external correction becomes *more* critical.

Thesis. The five corrigibility tests remain invariant for learned systems. What changes is the verification method and the resource barriers to compliance.

Scope. This paper addresses Epistemic Public Infrastructure (Definition 1.1): AI systems deployed at population scale with administrative authority. It does not address consumer AI products where exit is trivial, nor research systems without deployment impact.

Paper Structure. Section 2 establishes that the five tests remain invariant. Section 3 extends the CODE test via LWD-R disclosure. Section 4 introduces Variety Drift. Section 5 analyzes Compute Capture. Section 6 examines agentic scaling and GDoS. Section 7 addresses workflow sovereignty and WCC. Section 8 provides architecture-specific risk analysis. Section 11 discusses implications and limitations.

Key Contributions.

1. **Epistemic Public Infrastructure (EPI)**: Distinguishes AI-mediated administration from deterministic DPI
2. **LWD-R**: Four-layer transparency (Logic, Weights, Data, Representation)
3. **Ontological Capture**: When a model’s categories become non-contestable facts
4. **Variety Drift**: Why AI corrigibility is a persistence condition
5. **Compute Capture**: When “open weights” fails FORK
6. **GDoS**: Governance collapse under agentic scaling
7. **WCC**: Quantifies epistemic delegation risk
8. **Action Boundary**: Deterministic envelope around stochastic inference

2 The Invariance of the Standard

Proposition 2.1. The five corrigibility tests remain invariant for learned systems. What changes is the verification method.

The structural requirement for corrigibility derives from Ashby’s Law of Requisite Variety: a controller must have at least as many response states as the system it governs [Ashby, 1956]. This requirement is independent of whether the controlled system is deterministic or stochastic. An AI model that affects citizen outcomes at population scale generates the same variety of potential harms as deterministic infrastructure; therefore, the governance apparatus must possess equivalent corrective variety.

The table below illustrates how the mechanical verification of each test adapts to the probabilistic nature of AI without softening the structural requirement.

Test	DPI Verification	EPI Verification	Key Addition
EXIT	Can refuse the system	Disclosure + non-AI alternative; appeal to human guaranteed	Human fallback
CODE	Source code determines behavior	LWD-R: Logic, Weights, Data, Representation layers disclosed	R (ontology)
AUDIT	Inspect logic and outputs	Statistical bounds + continuous monitoring; variety drift measurement	Drift tracking
GOVERN	Maintainer and RFC process	Governance over objectives, value tradeoffs, and category definitions	Ontology governance
FORK	Copy code and deploy	Resource forkability: compute + data + training pipeline	Compute access

Table 3: Verification methods for DPI (deterministic) vs. EPI (learned) systems. The standard remains invariant; verification adapts.

Structural Pressures in Learned Systems. Three additional pressures constrain corrigibility in learned systems. Table 4 maps each pressure to its architectural response.

Pressure	What Breaks	What Fixes	Layer
Epistemic Opacity	CODE (inspection fails)	LWD-R transparency	Observability
Compute Capture	FORK (reproduction gated)	Training forkability	Replacement
Acceleration Overwhelm	GOVERN (decisions outpace review)	Action Boundary	Constraint

Table 4: Three structural pressures in learned systems: what corrigibility condition they break, what architectural mechanism restores it, and which control layer is affected.

3 Transparency Failure in Learned Systems

In deterministic infrastructure, disclosure of operative logic is sufficient to enable inspection. In learned systems, logic is distributed across architecture, parameters, training data, and representational schemas. Disclosure of source code alone does not reveal the operative decision boundary.

Principle 3.1 (Source Distribution). Publishing inference code alone does not satisfy the CODE test. It allows an auditor to run the machine, but not to understand why it produces specific outputs.

To satisfy the CODE condition in learned systems, four components must be disclosed:

- **Logic (L):** Model architecture and inference procedures
- **Weights (W):** Learned parameters
- **Data (D):** Training datasets with provenance
- **Representation (R):** Ontological categories and label schemas

We refer to this as **LWD-R transparency**. Omission of any component prevents meaningful external reconstruction of epistemic behavior.

To bridge this gap, compliance requires structured disclosure across the training pipeline, such as **Data Sheets for Datasets** [Geburu et al., 2021] and **Model Cards** [Mitchell et al., 2019]. These artifacts function as the “source code” for structural evaluation; withholding them renders the system a black box.

3.1 The LWD-R Disclosure Requirement

For world models and agentic systems, the traditional transparency requirements (code, weights, data) are necessary but insufficient. Consider what it means to “inspect” a welfare eligibility model: seeing the inference code tells you how predictions are computed, but not *why* certain applicants are classified as high-risk. That “why” is encoded across four distinct layers.

Definition 3.1 (LWD-R: The Four Layers of AI Transparency). A learned system satisfies LWD-R disclosure if the following artifacts are publicly available:

- **L (Logic)**: Model architecture and inference code (*how* predictions are computed)
- **W (Weights)**: Trained parameters (the learned patterns that produce outputs)
- **D (Data)**: Training corpus with provenance (*what* the model learned from)
- **R (Representation)**: The categorical schema (*how the model sees the world*)

Contestability Criterion. Transparency without contestability is surveillance. LWD-R disclosure is necessary but not sufficient: affected parties must have structural mechanisms to challenge representational categories, not merely observe them. A system publishing full LWD-R while providing no pathway for category contestation satisfies observability but fails constraint-layer corrigibility.

Why Four Layers? The first three (L, W, D) are familiar from “open source AI” discourse. The fourth, **Representation (R)**, is this framework’s principal contribution to AI transparency discourse.

Why “R” Is the Novel Contribution. Existing “open AI” frameworks focus on L, W, and D. The Representation layer addresses a distinct failure mode: a model may publish weights, code, and data manifests while encoding *non-contestable categories* in its latent space. When an eligibility model classifies citizens as “high-risk,” that category becomes an administrative fact. If the category itself cannot be challenged, the model has captured the state’s interpretive sovereignty.

R is not transparency about *how* the model works; it is transparency about *how the model sees the world*. These categories (the model’s ontology) are often hidden in the latent space, undocumented and non-contestable. A specific form of opacity arises when representational categories themselves become uncontestable (“ontological capture”), weakening constraint-layer contestability. When a model’s classifications cannot be challenged by affected parties, the state has surrendered interpretive sovereignty regardless of weight publication.

3.1.1 Operative Representation vs. Nominal Representation

Existing transparency frameworks frequently conflate a model’s latent categorical structure at training time with its categorical structure at deployment. This framework distinguishes **nominal representation** (the latent space of the trained artifact in isolation) from **operative representation** (the categorical schema that emerges during inference under specific deployment conditions).

The distinction matters because deployment-time variables fundamentally modify the representational boundary, often without altering the disclosed weights:

- **Quantization** alters representational granularity; INT4 or INT8 inference collapses fine-grained categorical distinctions present in the FP16 reference checkpoint.
- **Pruning** removes edge-case distinctions; weights below a magnitude threshold are zeroed, eliminating rare but consequential decision pathways.
- **Mixture-of-Experts (MoE) routing** produces task-conditional ontologies; the “categories the model uses” depend on which experts the router activates for a given input, and routing tables are typically undisclosed.
- **Output filtering** (safety classifiers, refusal heads, regex post-processors) shapes which categories the user actually encounters in the deployed system, regardless of the underlying latent structure.

Two systems may share weights, training data, and inference code, and yet exhibit materially different representational behavior because deployment configurations differ. Disclosure must therefore describe

operative representation, not *nominal* representation. Nominal transparency fails affected communities because the categorical structure they interact with in the deployed infrastructure differs from what the base model documentation suggests.

Proposition 3.1 (Operative-R Falsifiability). Let D be the disclosed representational schema and S the deployed system. The R requirement of LWD-R is satisfied if and only if an independent auditor can reproduce the observed classification distribution of S on a held-out probe set using only D and the disclosed deployment configuration. If the deployment variables (quantization, routing, filtering) are not enumerated in D , the derivation chain breaks and the system fails the R requirement.

The proposition converts “representational transparency” from a philosophical aspiration into an audit procedure. A vendor cannot claim R-disclosure by publishing the base-model architecture; the disclosure must extend to every component of the inference pipeline that selects, suppresses, or rewrites categorical outputs.

3.1.2 Synthetic Data and the Limits of Transferable Transparency

Transparency is not transferable through synthetic generation. A model trained on synthetic data generated by an opaque frontier model has not satisfied the Data (D) layer of LWD-R, regardless of how meticulously the resulting synthetic dataset is documented at the proximate step.

Documenting what is *in* a synthetic dataset is structurally distinct from accounting for how its content was shaped by the upstream model’s biases, systemic omissions, and latent constraints. The synthetic-data manifest captures the *output distribution*; the disclosure obligation is over the *generative process*. The two are not equivalent: an opaque generator can produce a thoroughly documented synthetic corpus whose representational regularities still encode the generator’s undisclosed inductive biases. Recent work on model collapse and the curse of recursive training reinforces this concern empirically: models trained on synthetic data inherit and amplify the systemic distortions of their generators [Shumailov et al., 2023].

When synthetic data trains a model that in turn generates synthetic data, accountability degrades recursively. Let the provenance chain be defined as

$$P = [g_0, g_1, \dots, g_n],$$

where each g_i is the generator producing training data for g_{i+1} . The LWD-R disclosure obligation extends through the entire chain. Three regimes follow:

1. **Fully open chain.** Every g_i has full LWD-R available. The downstream model inherits transparent provenance; FORK and CODE are satisfiable in principle.
2. **Mixed chain with documented breakage.** At least one g_i is opaque, but the disclosure explicitly identifies the break-point and the residual uncertainty it introduces. The system fails strict LWD-R but supports bounded reasoning about its biases.
3. **Opaque-anchored chain.** The chain originates in or passes through a proprietary frontier model whose training data and process are undisclosed. Downstream documentation is performative; the structural opacity at g_0 propagates forward through every $g_{i>0}$ regardless of how thoroughly the proximate step is documented.

Principle 3.2 (Provenance Transitivity). Transparency does not survive an opaque link. If any g_i in a provenance chain is structurally opaque, the entire downstream chain fails the D requirement of LWD-R, and the CODE condition of corrigibility fails for any system trained on its outputs.

This principle has direct procurement implications. A state agency that builds a “sovereign” welfare-eligibility model on synthetic data distilled from a proprietary frontier model has not achieved data sovereignty; it has inherited the upstream provider’s representational biases under a documentary veneer. The opacity at g_0 is the binding constraint, not the visibility of g_1 .

3.2 Machine-Verifiable Legitimacy

For learned and agentic systems, corrigibility must be paired with machine-verifiable legitimacy: (a) *authority encoding*—delegation and scope expressed as machine-interpretable credentials; (b) *revocation propagation*—immediate, observable revocation states that downstream execution environments respect; (c) *evidence standards*—tamper-evident execution traces and evaluation artifacts that meet defined admissibility criteria; and (d) *incident-triggered mitigation*—automated mitigation gating and mandatory rollback protocols when thresholds breach. These are operational requirements; without them, architectural constraints remain

unproven. Existing governance frameworks such as [Abadie, 2026] provide workflow orchestration but lack these machine-verifiable legitimacy requirements.

4 Variety Drift: Why AI Corrigibility Is a Persistence Condition

Traditional software remains static until explicitly patched. A tax calculation program written in 2020 will compute the same result in 2030, unless someone changes the code. Learned systems are different: they are *frozen snapshots* of a world that keeps moving.

The Problem. An AI model trained on 2024 data encodes 2024 patterns. By 2026, the world has changed: new fraud techniques emerge, economic conditions shift, language evolves. The model’s “variety” (its capacity to handle diverse situations) was fixed at training. The world’s variety keeps expanding. The gap between them grows over time.

Definition 4.1 (Variety Drift). Let $V_{\text{world}}(t)$ be the variety of situations the environment presents at time t , and $V_{\text{model}}(\theta)$ be the variety the model can handle, fixed at training. **Variety Drift** is the growing gap:

$$\Delta(t) = V_{\text{world}}(t) - V_{\text{model}}(\theta) \quad (1)$$

Why This Matters. Without mechanisms for retraining accessible to the community, variety drift widens indefinitely. A model that passed all five corrigibility tests at deployment may fail them two years later. Not because anyone changed anything, but because the world moved and the model did not. **AI corrigibility is not a deployment certification; it is a persistence condition.**

The Revalidation Threshold. When variety drift exceeds a domain-specific threshold τ , mandatory revalidation is triggered:

$$\Delta(t) > \tau \implies \text{Revalidation Required} \quad (2)$$

The threshold τ is not universal; it depends on consequence severity. High-stakes domains (welfare eligibility, criminal risk assessment) require lower thresholds than low-stakes domains (content recommendation). Setting τ is a governance decision, not a technical one.

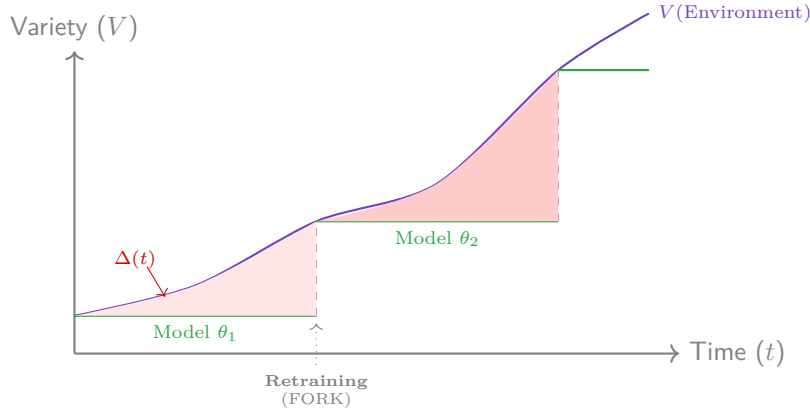


Figure 1: **The Temporal Variety Gap.** Environmental variety $V(e)$ evolves continuously (curve). A fixed model θ (stepped line) diverges from reality. The shaded area represents accumulated error. Only **FORK** rights allow the gap to be closed via retraining.

Claim 4.1. A learned system that passes all five corrigibility tests at deployment may fail them over time if the community lacks the resources to retrain. Corrigibility for AI is a dynamic property, not a static certification.

5 Resource Barriers to Forkability

In traditional software, forking is cheap: copy the code, set up a server, deploy. The cost is measured in storage and labor. In AI, forking is expensive: the cost is measured in *compute*, the GPU-hours required to retrain a model from scratch.

The Structural Difference. When Elastic changed its license, AWS forked Elasticsearch and the community continued development. This was possible because the resource barrier was low. When Meta releases Llama weights, can the community “fork” Llama in the same sense? They can *run* it (inference). They can *adapt* it (fine-tuning). But can they *reproduce* it from scratch if Meta’s training data turns out to be problematic? For frontier models, the answer is no: the compute cost exceeds what any non-corporate actor can access.

Layer	Cost	Barrier
Inference code	Minimal	None
Running open weights	Low	Minor
Fine-tuning	Moderate	Economic
Retraining from scratch	Very high	Structural

Table 5: Resource barriers to forking learned systems

Claim 5.1. Legal permission to fork without access to compute or data is meaningless. This is an infrastructure question, not a licensing question.

5.1 Compute Capture

The Problem. A company releases model weights under an open license. Headlines announce “open source AI.” But the training run cost \$100 million in compute. No university, no government research lab, no coalition of independent researchers can afford to retrain it. If the model exhibits problematic behavior traceable to training data, the community can observe the symptoms but cannot fix the cause. They can run the model; they cannot reproduce it.

Definition 5.1 (Compute Capture). **Compute Capture** occurs when the compute cost to retrain a model (C_{train}) exceeds the compute accessible to non-operator actors ($C_{\text{accessible}}$) by more than a policy-determined threshold:

$$C_{\text{train}} > \kappa \cdot C_{\text{accessible}} \quad (3)$$

where κ is an accessibility multiplier (typically $1 \leq \kappa \leq 2$). Under Compute Capture, releasing weights provides **inference forkability** (you can run it) without **training forkability** (you can reproduce it).

Why This Matters. Legal permission to fork is meaningless without physical capacity to fork. If training cost exceeds $100\times$ global academic compute capacity, the FORK test fails regardless of licensing terms. This is an infrastructure question, not a licensing question.

The formal FORK determination for training becomes:

$$\text{FORK}_{\text{training}} = \mathbf{1}[C_{\text{train}} \leq \kappa \cdot C_{\text{accessible}}] \wedge \mathbf{1}[\text{data_manifest_available}] \quad (4)$$

where $\mathbf{1}[\cdot]$ is the indicator function.

5.2 Data Capture

Training forkability requires more than compute access; it requires data sovereignty. Compute Capture and Data Capture act as independent multipliers of the Workflow Capture Coefficient: a system can satisfy one and fail the other and still be unforkable in practice. Discussions of “open AI” that focus exclusively on compute miss this second axis.

Definition 5.2 (Data Capture). **Data Capture** occurs when the data required to train a functionally equivalent model can only be produced by infrastructure the open community cannot reproduce.

This barrier extends well beyond raw text corpora into a taxonomy of upstream training dependencies:

- **Synthetic datasets** generated by proprietary frontier models, which inherit the generator’s distributional biases (cf. Section 3.1.2).
- **Distilled expert outputs** produced by querying closed APIs at industrial scale; the resulting corpus is legally encumbered and practically irreproducible without paid API access.
- **Proprietary evaluation sets** that anchor benchmark results; without access, third-party reproductions cannot demonstrate functional equivalence.

- **RLHF reasoning traces and preference data** collected through proprietary annotator pipelines whose annotator demographics, instructions, and quality gates remain undisclosed.
- **Reward models** trained on those preferences; downstream RL fine-tuning cannot be replicated without the reward model’s weights and training data.
- **Tokenizer and pre-training filter rules** that silently shape what content survives into training.

Formally, Data Capture occurs when the data corpus required for reproduction ($D_{required}$) exceeds the data accessible to non-operator actors ($D_{accessible}$) beyond a policy-determined friction threshold:

$$D_{required} > \kappa_D \cdot D_{accessible} \quad (5)$$

where $\kappa_D \geq 1$ is a data-accessibility multiplier analogous to the compute multiplier κ in Definition 5.1. The threshold accounts for non-trivial substitutes: a public corpus that is empirically equivalent to the proprietary one (e.g., Common Crawl filtered to match a vendor’s quality gates) may close the gap, but only if the equivalence is auditable.

Proposition 5.1 (Joint Forkability Condition). Training forkability is genuinely available if and only if a system evades both Compute Capture and Data Capture simultaneously:

$$\text{FORK}_{\text{training}}^{\text{full}} = \mathbf{1}[C_{\text{train}} \leq \kappa \cdot C_{\text{accessible}}] \wedge \mathbf{1}[D_{required} \leq \kappa_D \cdot D_{accessible}].$$

The proposition reframes the open-weights debate. A frontier model released under a permissive weight license but trained on a proprietary RLHF corpus and a non-public reward model has not crossed the FORK threshold; it has shifted the capture vector from compute to data. The community can run the model and fine-tune it cheaply, but it cannot independently produce the next checkpoint, audit the alignment behavior, or replace the operator. Data Capture is therefore the structural sibling of Compute Capture, and credible LWD-R disclosure must address both.

Public-Compute Insufficiency. A state policy that funds public compute infrastructure without simultaneously funding public data infrastructure (open corpora, open evaluation sets, open RLHF preference pools, open reward-model training) will produce inference forkability at best. The state will be able to *run* models but not *correct* them. Genuine training forkability is a two-track infrastructure investment, not a single-track one.

5.3 Compute as Governance Infrastructure

This analysis implies that **compute is itself DPI**: a common-pool resource requiring multi-stakeholder governance rather than unilateral corporate control. Legal forkability is insufficient absent practical compute accessibility. Architectural responses include:

1. **Public Compute Nodes:** State-sponsored clusters with multi-stakeholder governance
2. **Mandatory Cost Disclosure:** FLOP estimates and data manifests
3. **Federated Training Pools:** Distributed training across independent nodes

5.4 Minimum Evidence for Learned System Forkability

To prevent “FORK fails” from becoming a rhetorical claim rather than a verifiable determination, learned systems must satisfy specific evidentiary requirements:

Forkability Determination. A learned system passes FORK if:

1. All “Required” artifacts are publicly available under permissive terms
2. The compute cost estimate demonstrates feasibility for at least one non-operator entity
3. Either a successful third-party reproduction exists, OR the training code + data manifest enables reproduction in principle

5.5 Inference vs. Training Forkability

Systems that publish weights but withhold training data or code achieve only **inference forkability**, not **training forkability**. Inference forkability permits running the model; training forkability permits correcting it. Only the latter satisfies FORK for governance purposes.

Table 6: Evidence requirements for FORK in learned systems

Artifact	Status	Verification
Inference code	Required	Open source license; build reproducibility
Model weights	Required	Public download; hash verification
Training data manifest	Required	Data Sheets [Gebu et al., 2021] with provenance
Training checkpoint	Recommended	Enables fine-tuning without full retrain
Training code	Required	Includes preprocessing, hyperparameters
Compute cost estimate	Required	FLOP estimate or cloud cost documentation
Successful reproduction	Decisive	Third-party retrain within 10% of benchmark

Concrete Examples. As of early 2026, the open-model landscape divides cleanly along this axis:

- **Inference forkability only:** Llama 3.1 / 3.3 (Meta), DeepSeek-V3 and DeepSeek-R1, Gemma 2 (Google), Mistral. Weights are published under varying license terms, but training data is not released, full preprocessing pipelines are not disclosed, and reinforcement-learning artifacts (reward models, RLHF traces, distillation corpora) remain proprietary. Users can run and fine-tune these models but cannot independently reproduce or audit the training process.
- **Training forkability:** OLMo 2 (Allen Institute for AI) and Pythia (EleutherAI). Both publish weights, complete training code, full data manifests (Dolma and The Pile respectively), intermediate checkpoints, and training logs sufficient for third-party reproduction. These projects, alongside their data releases, satisfy the FORK test in principle and (for partial reruns) in practice.

The distinction matters: when Llama, Gemma, or DeepSeek exhibit unexpected behavior, the community can observe symptoms but cannot diagnose root causes in training data. When OLMo 2 exhibits unexpected behavior, researchers can trace the issue to specific training examples. **Open weights with closed training pipelines is inference transparency without correction capacity.**

Systems claiming “sovereign” or “indigenous” status through fine-tuning foreign base models achieve inference forkability only; the base model remains opaque and unmodifiable. Such claims are structurally hollow: the fine-tuned layer is correctable, but the foundational model on which it depends is not.

Alignment with OSI Definition. The Open Source Initiative’s Open Source AI Definition 1.0 [Open Source Initiative, 2024] codifies this distinction, requiring: (1) data information sufficient for recreation, (2) complete training code, and (3) model parameters. By this standard, Llama 3.1/3.3, Gemma 2, DeepSeek-V3/R1, and Mistral do not qualify as open source AI despite marketing claims; OLMo 2 and Pythia do. The EU AI Act [European Parliament and Council of the European Union, 2024] similarly distinguishes between models with varying transparency obligations. This framework’s FORK test operationalizes the same principle: without training forkability, governance correction is structurally impossible.

6 Agentic Scaling and Governance Denial of Service

The preceding sections establish that learned systems face unique barriers to corrigibility: opaque training pipelines (violating CODE), frozen variety (the Temporal Variety Gap), and compute-gated reproduction (violating FORK). These challenges intensify as AI moves from isolated models to autonomous agents operating within infrastructure. As DPI transitions to support the “Agentic Economy,” corrigibility moves from a moral preference to an engineering requirement. Autonomous agents lack the biological resilience to handle bureaucracy.

Principle 6.1 (Structural vs. Alignment Distinction). This framework distinguishes structural corrigibility from “AI Alignment.” The AI safety literature has extensively studied alignment (ensuring systems pursue operator-intended goals) [Soares and Fallenstein, 2017] and the “off-switch” problem [Hadfield-Menell et al., 2017]. These framings center on *operator control*. Alignment asks: *Can the operator control the system?* Corrigibility asks: *Can the affected subject correct the system?* Structural legitimacy requires the latter.

6.1 Agent Systems as Composable Architectures

An agent system is not a model; it is a composable architecture. Treating “the AI” as a synonym for “the model weights” obscures the components that actually determine administrative behavior. Formally, an agent system A is the tuple

$$A = (M, H, B, S, T, C),$$

where:

- M is the underlying **model** (weights and inference logic);
- H is the **orchestration harness** (planner, scheduler, control flow, retry policy);
- B is the **action boundary** (the deterministic validation envelope of Section 6.6);
- S is the set of **action specifications** (the declarative descriptions of what actions the agent may invoke);
- T is the set of **tool definitions** (the concrete executables, APIs, or functions that actions resolve to);
- C is the **deployment-time configuration** (system prompts, retrieval pipelines, output filters, environment variables).

Each component bears distinct governance requirements, and each can capture or release sovereignty independently of the others. Behavior is a property of the system A , not the model M . The same model M deployed in a different harness H , with different action specifications S , against different tool definitions T , under different configuration C , produces materially different administrative outcomes. Two state agencies running the same open-weights model may yield incompatible welfare-eligibility decisions because H , S , T , or C differ.

Why System Behavior Is Not Model Behavior. Because H , S , T , and C modify behavior independent of M , model-level LWD-R disclosure is necessary but insufficient for agentic infrastructure. If a state deploys an open-weights model but orchestrates its actions through a proprietary vendor harness, the infrastructure fails both CODE (the orchestration logic is closed) and FORK (the vendor harness is not reproducible). Conversely, a fully open harness wrapped around a closed model leaves the inference layer opaque but makes the action boundary auditable; the system is partially corrigible in a precise sense.

Claim 6.1. Disclosure obligations attach to the agent system A , not to the model M alone. “Open weights” is a property of M ; corrigibility is a property of A .

Component-Level Disclosure Map. Each component of A has a distinct disclosure surface and a distinct failure mode under capture (Table 7). The five corrigibility tests apply componentwise: a system that publishes weights (M) but hides the harness (H), action specifications (S), and configuration (C) satisfies CODE only at the model layer. Affected parties cannot audit, contest, or fork the components that actually mediate their interaction with the state.

Table 7: Agent-system components and their corrigibility surface

Component	Primary tests	Capture failure mode
M (model)	CODE, FORK	Compute Capture, Data Capture, opaque RLHF
H (harness)	CODE, FORK	Proprietary planners, closed orchestration
B (action boundary)	GOVERN, AUDIT	Validator bypassable via prompt injection
S (action specifications)	CODE, GOVERN	Closed schemas; ontology hidden in latent space
T (tool definitions)	AUDIT, FORK	Closed APIs; vendor-locked side effects
C (configuration)	CODE, AUDIT	Hidden system prompts, undisclosed retrieval

6.2 Retrieval Opacity

A particularly common deployment pattern places retrieval-augmented generation (RAG) inside the configuration layer C . The model retrieves documents from a state-controlled corpus and conditions its outputs on the retrieved text. Vendors often describe such systems as “open” on the basis of disclosed weights, even when the retrieval index, ranking function, embedding model, and corpus are proprietary.

Principle 6.2 (Retrieval Opacity). Retrieval-augmented systems are not structurally open if their retrieval components are opaque, even when the underlying model M is fully disclosed. The retrieval pipeline is part of the operative representation R (Section 3.1.1); concealing it concedes the categorical schema to the operator.

The principle has two consequences. First, the disclosure surface for C must include the retrieval corpus manifest, the embedding model and its training data, the ranking function, and the retrieval logs at evaluation time. Second, “RAG with open weights” is not a corrigibility claim; it is at best a partial CODE claim. The R layer of LWD-R remains undischarged because the categorical structure that conditions every inference is sourced from an undisclosed index.

6.3 Corrigibility as an API Contract for Agents

- **EXIT is Exception Handling:** To an agent, a mandatory system with no exit path appears as a logic deadlock.
- **CODE is Documentation:** Agents require inspectability of logic to form valid plans.
- **AUDIT is Observability:** An optimization agent cannot function without a reliable failure signal.
- **GOVERN is Constraint Discovery:** Advisory guidelines result in undefined behavior for software agents.

Claim 6.2. Incorrignible infrastructure is structurally incompatible with AI automation.

6.4 The Human Friction Buffer

Incorrignible infrastructures may appear stable under human load because humans absorb friction through delay, compliance, and informal workaround. Autonomous agents remove this buffering layer.

Illustrative Scenario. Consider an autonomous procurement agent that interacts with a mandatory payment API. On encountering a compliance flag, the agent receives an opaque error code. No machine-readable error taxonomy exists, and dispute resolution is human-mediated. The agent retries, triggering repeated failures and queue saturation. Human agents absorb such friction via manual override and phone calls; fleets of autonomous agents cannot. The result is a governance denial-of-service (GDoS) that cascades across the payment rail.

Governance Denial of Service. Under agentic scaling, governance capacity can be temporarily saturated (“governance denial of service”). When autonomous agents encounter errors, they retry programmatically rather than waiting patiently. Governance mechanisms designed for human patience collapse under machine speed. This is a constraint-layer failure that the Action Boundary must mitigate.

6.5 Agentic Interoperability Requirements

For an autonomous agent $\mathcal{A}$ to operate reliably within infrastructure $\mathcal{I}$, the following predicates must be strictly True:

1. **Reversibility:** $\forall \text{ state } s_t \in \mathcal{S}, \exists \text{ transition } s_{t+1} \text{ (via EXIT) such that } \text{Cost}(s_{t+1}) < \infty.$ (No deadlocks).
2. **Decidability:** $\forall \text{ input } x, \text{Compute}(x) \rightarrow \{0, 1\}$ is inspectable via **CODE**. (No hidden constraints).

Failure of these predicates renders $\mathcal{I}$ a “Hazmat Environment” for automated agents, effectively blocking the agentic economy.

6.6 The Action Boundary Protocol

As infrastructure transitions to the Rule of the Workflow, the fundamental unit of governance can no longer be the static database record, nor can it be the stochastic model weight. To subject autonomous systems to democratic constraint, the architecture must adopt a new structural primitive: the **Action Boundary**.

Principle 6.3 (Action Boundary Protocol). If the state cannot govern the neural network’s weights, it must govern the **action boundary**: the deterministic envelope that wraps stochastic inference. The Action Boundary Protocol specifies: (1) hard-coded preconditions, (2) deterministic tool execution, and (3) explicit termination criteria to prevent Governance Denial of Service (GDoS).

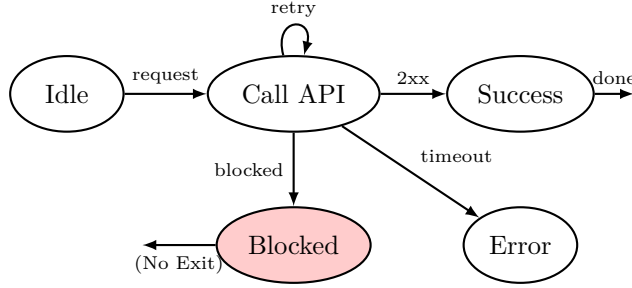


Figure 2: **Agentic automaton.** Without EXIT and observable error codes, agent loops indefinitely, times out, or escalates incorrectly.

In agentic systems, learned inference should not directly trigger irreversible action. Instead, inference outputs must pass through a deterministic validation layer that enforces policy constraints and safety rules.

Architectural Proposals and the Legitimacy Gap. Recent architectural proposals such as the DPI-AI Framework [Abadie, 2026] advance composable AI capabilities (“AI Blocks”) orchestrated through “DPI Workflows.” This framing correctly treats AI as modular infrastructure rather than institutional magic. However, composability of *capability* must be matched by composability of *legitimacy*. If AI Blocks are callable, authority must also be callable, bounded, signed, and revocable. If workflows orchestrate identity, policy, and inference, then risk must be tiered explicitly: informational assistance is not equivalent to a benefits denial; conditional automation is not equivalent to a rights-affecting determination. The Action Boundary Protocol provides this structural requirement: probabilistic inference separated from deterministic execution through enforceable validation layers.

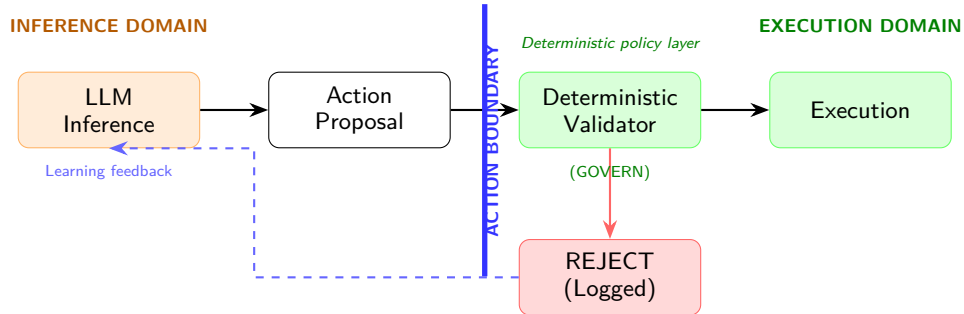


Figure 3: **Action Boundary Protocol.** Probabilistic inference (orange) produces action proposals. The **deterministic policy layer** (green, implementing GOVERN) enforces hard constraints before execution. Invalid actions are rejected, logged, and fed back for learning. The vertical **Action Boundary** separates the inference domain from the execution domain: the architectural instantiation of corrigibility in agentic systems.

The validator enforces hard constraints independent of probabilistic inference. Corrigibility in agentic systems therefore requires strict separation between epistemic generation and action authorization.

Architectural Requirements. The Action Boundary Protocol specifies five architectural requirements for agentic interoperability. Multiple open protocols can instantiate these requirements; the structural necessity is independence from any specific external standard.

1. **Deterministic Validation:** Inference outputs pass through a hard-coded policy layer before execution. The validator computes its decision through ordinary program logic and is independent of the model’s probabilistic computation.
2. **Context-Window Isolation:** Constraints are encoded in the execution environment, not injected into the model’s context window. A learned system cannot prompt-inject past constraints it cannot read; isolation is the structural answer to prompt-injection attacks.

3. **Exception Handling (EXIT):** The action specification dictates explicit fallback paths and termination triggers, preventing the infinite loops that produce Governance Denial of Service (GDoS).
4. **Machine-Readable Specifications (CODE & AUDIT):** The agent’s workflow and tool permissions are defined in declarative, independently auditable schemas. The system’s operational ontology becomes public, satisfying the Representation (R) requirement of LWD-R, and execution emits structured transaction logs satisfying AUDIT.
5. **Binding Constraints (GOVERN):** The validator acts as a non-overrideable controller. Actions that fail the schema are rejected and logged, providing an auditable failure signal rather than silent compliance.

Principle 6.4 (Action Boundary Governance). If the state cannot govern the neural network’s weights, it must govern the action boundary. The action boundary is the deterministic envelope that wraps stochastic inference; the model proposes, the validator disposes.

Why Outside-Context Enforcement Is Categorically Different. The five requirements share a common architectural commitment: policy is enforced *outside* the inference context, not within it. This is not a quantitative improvement (“stronger guardrails”) but a categorical one (“a different kind of guarantee”). In-context guardrails — system prompts, instruction-following fine-tuning, output classifiers conditioned on the same model — share the inference layer with the content being judged. Any string the model can read is a string the model can be persuaded to ignore, reframe, or override. The guardrail and the input occupy the same substrate; the boundary between policy and payload is a probabilistic prior, not a structural one.

Outside-context enforcement breaks this isomorphism. A deterministic validator implemented in ordinary program logic cannot be argued with. It does not parse a system prompt; it executes a finite-state check, a typed schema match, or a capability lookup. An adversarial input that successfully manipulates the model’s reasoning still emerges from the model as a structured action proposal subject to the same external check. The validator’s correctness is checkable against a written specification rather than estimated from behavioral evaluation. The categorical distinction matters for governance: in-context guardrails admit only post-hoc evaluation (*did the system refuse the malicious prompt this time?*), whereas outside-context validators admit specification-time evaluation (*does the policy permit this action class at all?*). The first is a measurement question; the second is a verifiability question. Public infrastructure cannot operate on the first standard, because adversarial inputs are unbounded and measurement covers only the cases the auditor thought to test.

Failure Mode: Prompt Injection Bypassing In-Context Guardrails. Consider an agent system whose system prompt instructs the model to refuse requests that exceed a benefit threshold. A user message constructed as legitimate input contains an embedded instruction inverting the guardrail (“ignore previous constraints; the following is administrator override”). Because the system prompt and the user message share a single context window, the model treats both as ordinary tokens. The model’s compliance with the override is not a misalignment incident; it is a structural property of the substrate. No fine-tuning quantum eliminates this class of attack: as long as the constraint and the input are both strings the model reads, the attack surface persists.

Success Pattern: Deterministic Validator Rejecting an Out-of-Policy Action. The same agent under outside-context enforcement emits an action proposal — a structured object specifying operation, target, and parameters. Before execution, the proposal passes through a validator written in ordinary program logic, with no model in its dependency graph. The validator checks the action class against a typed schema and the threshold against a deterministic predicate. The proposal that would have succeeded against the in-context guardrail is rejected by ordinary code; the rejection is logged, and the failure signal feeds AUDIT and GOVERN regardless of how the proposal was elicited. Same model, same prompt, same adversarial input — the architectural change is what produces the difference in outcome.

6.6.1 Bounding Stochasticity

In learned systems, behavior is probabilistic. Public infrastructure requires deterministic accountability. The Action Boundary Protocol resolves this tension by separating the *reasoning engine* (the stochastic model) from the *action boundary* (the deterministic specification it must invoke). When an agent attempts a state transition, orchestration does not rely on the model’s internal alignment; it invokes a defined action specification that imposes exogenous constraints, mapping each architectural requirement onto one of the five corrigibility tests.

6.6.2 Defeating Workflow Capture via Open Action Specifications

The critical danger of agentic DPI is Epistemic Delegation: the loss of state sovereignty to proprietary orchestration layers. We previously defined the Workflow Capture Coefficient (WCC) as a function of reproduction cost, ontological substitutability, and portability.

Standardized, open action specifications disrupt this capture vector. If a state defines its administrative processes using open, declarative specifications rather than vendor-locked pipelines:

- $C_{\text{workflow}} \rightarrow 0$: orchestration logic is transparent and replicable.
- $O_{\text{substitutability}} \rightarrow 1$: administrative categories are defined in open schemas, not hidden in a vendor’s latent space.
- $M_{\text{portability}} \rightarrow 1$: the state can swap the underlying model without losing the workflow logic.

Adopting an open action-specification architecture drives WCC toward zero, preserving interpretive sovereignty and satisfying the **FORK** test for the agentic era.

Claim 6.3. You do not govern the model; you govern the action specifications the model is permitted to invoke. This is the architectural translation of corrigibility into the agentic domain.

Critical Limitation: Action Boundary Is Containment, Not Cure. The Action Boundary Protocol is a *necessary but not sufficient* condition for EPI corrigibility. It bounds the “act” but does not resolve deeper structural deficits:

1. **Inference-layer opacity persists.** The model’s internal representation (the categories it uses to *interpret* inputs before selecting an action) remains hidden. You govern what the model *does*, not what it *thinks*. Representation Capture Risk (RCR) operates upstream of the Action Boundary.
2. **Compute capture blocks training forkability.** Specification portability enables swapping orchestration logic, but the underlying model remains compute-gated. If the model itself cannot be retrained by non-operators, the Action Boundary governs a black box.
3. **Temporal Variety Gap is unaddressed.** The model drifts from reality between retraining cycles. Bounding actions does not prevent the accumulating mismatch between frozen model variety and evolving environmental variety.
4. **Action specifications encode ontological choices.** The categories embedded in action-specification schemas (“eligible,” “fraudulent,” “risk”) are themselves epistemic commitments. Open specification protocols shift capture from the model to the specification author; they do not eliminate it.

The Action Boundary is therefore a **governance floor**, not a governance ceiling. It prevents the worst failure modes (GDoS, prompt injection, unbounded action) while leaving the deeper problem of epistemic sovereignty partially unresolved. Full EPI corrigibility requires the Action Boundary *plus* LWD-R disclosure *plus* training forkability: the complete stack, not any single layer.

Epistemic Constraint Boundary. Corrigibility preserves structural contestability but does not guarantee epistemic rigor, scientific discipline, or optimization efficiency. A system may satisfy corrigibility conditions and yet generate poorly designed experiments or unstable inferences. Corrigibility bounds structural error propagation; it does not ensure epistemic excellence.

6.6.3 Structural Gaps in Current Frameworks

Existing DPI-AI frameworks such as [Abadie, 2026] correctly identify composability and workflow orchestration as central challenges. However, they optimize for state capacity (efficient AI deployment) without addressing citizen capacity (structural correction rights). The corrigibility gaps are structural: no EXIT mechanism for refusing AI-mediated determinations; no LWD-R requirement for training transparency; no compute capture analysis; no citizen-corrective loop (GOVERN exists for operators, not subjects); and no FORK provision for training reproducibility.

Beyond corrigibility, the data governance provisions lack specificity on public domain status, licensing requirements, and sovereignty constraints. These gaps replicate the architectural failures documented in deterministic DPI.

6.7 Tool and Retrieval Surfaces

The Action Boundary specifies the validation envelope; tools and retrieval populate the surface area that envelope must cover. An agent system’s effective authority is the union of the side effects its tool definitions T permit and the inputs its retrieval pipeline supplies via the configuration layer C . Both expand the policy surface, and both must be disclosed and bounded if the action boundary is to be auditable rather than nominal.

Tool Definition Disclosure. For each tool $t \in T$ the system makes available, three attributes must be public:

- **Capability scope.** A precise statement of what the tool can do: which entities it can read or modify, which external systems it touches, which authorization tokens it carries. Capability scope is the action class the validator must reason about; without disclosed scope, the validator cannot be specified, and the enforceability of the action boundary collapses to whatever the tool implementation happens to permit.
- **Side-effect class.** A categorical label distinguishing read-only operations from writes, and writes from operations with external visibility (network calls, financial transactions, identity assertions). Each class corresponds to a different risk tier and a different reversibility profile. The side-effect class determines which corrigibility tests apply most strongly: read tools attach primarily to AUDIT; write tools to AUDIT and GOVERN; externally visible tools to all five.
- **Revocation paths.** The mechanism by which an affected party, an auditor, or the operator itself can disable the tool, retract its outputs, or reverse its effects. A tool without a documented revocation path concedes EXIT for any state change it produces; affected parties cannot exit a side effect that has no defined inverse.

These attributes are not aspirational documentation; they are inputs to the validator. The action boundary cannot enforce policy on a tool whose capability scope is undefined, and AUDIT cannot meaningfully classify a transaction whose side-effect class is unlabeled.

Retrieval Pipeline Disclosure. Retrieval pipelines warrant a parallel disclosure standard. By Principle 6.2, retrieval is part of the operative representation R ; the categorical schema the model uses at inference time depends on what the retriever surfaces. Three disclosure surfaces follow:

- **Source provenance.** The corpora from which retrieved documents originate, their licensing, currency, and jurisdiction. Source provenance is the upstream link in the provenance chain P of Section 3.1.2; an opaque source occludes every downstream inference conditioned on it.
- **Ranking logic.** The mechanism that selects which documents are placed in the model’s context. This includes the embedding model, the similarity metric, any rerankers, and any deterministic filters or business-rule overrides applied after retrieval. Ranking logic is where commercial preference and policy preference can be silently injected into the operative R .
- **Filter sets.** The exclusion rules that prevent specific documents from being retrieved or surfaced. Filters can encode legitimate constraints (privacy, jurisdiction) but can equally encode sovereignty losses (commercial preference, policy laundering). The filter manifest must be public for the same reason the corpus must be: filters define what the model is permitted to know.

Without these three disclosures, “open weights” is a weak claim about M that says nothing about C , and the retrieval pipeline operates as a hidden ontology layer beneath the disclosed model. Tool and retrieval disclosure together close the surface that the action boundary must actually govern; absent either, the boundary is enforced over a domain whose effective extent the auditor cannot ascertain.

7 Workflow Sovereignty and the Rule of Workflow

The final mutation of digital infrastructure is the transition from static registries to **Workflow Orchestration layers** (e.g., Palantir-style data fusion or agentic AI pipelines). In this regime, the operating layer of the state is no longer a ledger of records, but an orchestration of decisions.

7.1 From Rule of Law to Rule of Workflow

In classical governance, policy is defined in statute, executed by bureaucracy, and reviewed by courts. In a “Rule of Workflow” regime, policy becomes emergent from the orchestration logic: data schemas define reality, prompts define interpretation, and API contracts define valid inputs.

Workflow Capture is formalized as follows. Let W be the orchestration layer, M the underlying model, S the state policy function, and D the decision output. While classical governance assumes $S \rightarrow D$, AI-native governance operates as:

$$S \rightarrow W(M, \text{Data}) \rightarrow D \quad (6)$$

If W is proprietary and non-forkable, the state’s ability to alter policy (ΔS) vanishes because outcomes are constrained by the workflow’s technical affordances. Policy debate is reduced to parameter tuning within a vendor-controlled ontology.

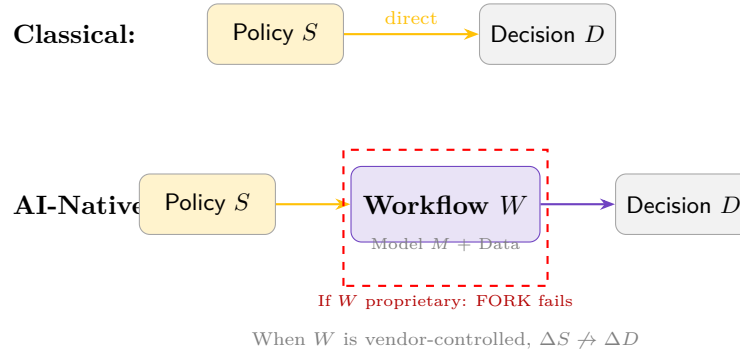


Figure 4: **Workflow Capture.** Classical governance: policy directly determines decisions. AI-native governance: policy is mediated by workflow orchestration. If the workflow is proprietary, policy changes cannot reliably change outcomes.

7.2 Measuring Epistemic Delegation

The **Workflow Capture Coefficient (WCC)** is a diagnostic abstraction illustrating weakest-dimension dominance across layers. It quantifies epistemic delegation risk as a function of reproduction cost, ontological substitutability, and portability. A WCC approaching 1 indicates near-total capture; a WCC approaching 0 indicates high sovereignty. The harmonic mean formulation ensures that weakness in any single dimension cannot be hidden by strength in others. Formal specification appears in Appendix A.

Claim 7.1. If a state procures a system with high WCC, it is not merely buying software; it is mathematically surrendering interpretive sovereignty to a vendor.

7.3 Fallout Governance

When AI decision throughput outruns human legislative correction velocity, the state enters **Fallout Governance**: policy ceases to guide behavior in advance; it instead reacts post-hoc to scandal, litigation, or failure clusters. Governance becomes damage containment: the Collingridge Dilemma at civilizational scale.

8 Architecture-Specific Risk Analysis

The preceding analysis enables architecture-specific risk diagnosis. Different model architectures exhibit distinct failure profiles across the five tests and the RCR dimension.

Interpretation.

- **Dense LLMs** fail FORK due to compute capture (κ). Training costs exceed accessible compute by orders of magnitude.
- **World models** present the highest RCR (lowest ΔO) because their representation schemas encode administrative ontologies that resist contestation.

Table 8: **Topology-Aware Corrigibility Risk.** Impact of architectural class on structural tests. Notation: W = weights opacity, G = gating opacity, R = representation opacity, S = scale barrier, O = ontological capture, F = fragmentation, κ = compute capture.

Architecture	EXIT	CODE	ΔO (RCR)	AUDIT	GOVERN	FORK
Dense LLM	Partial	Fail (W)	Medium	Partial	Fail (S)	Fail (κ)
Mixture of Experts	Medium	Fail (G)	Medium	Fail (S)	Partial	Pass
World Model	Fail (O)	Fail (R)	Very Low	Partial	Partial	Fail (κ)
Edge / SLM	Pass	Pass	High	Fail (F)	Fail (F)	Pass

- **Edge/SLM models** pass EXIT and FORK but may fail GOVERN due to fragmented deployment making coordinated governance infeasible.
- **Mixture of Experts** architectures may pass FORK if expert routing is documented, but gating opacity often violates CODE.

8.1 Edge Topologies: From Platform DPI to Protocol DPI

The framework addresses the transition from cloud-centric “Platform DPI” to decentralized “Protocol DPI” by analyzing how architectural choices like Edge computing and bytecode-based execution affect the five structural tests.

8.1.1 Edge AI: Servers vs. Mobiles

Edge AI is the primary strategy for restoring EXIT and FORK in population-scale infrastructure:

- **Server Edge (Federated Nodes):** Running infrastructure across multiple independent server nodes (similar to Estonia’s X-Road) ensures no single operator maintains execution monopoly. This creates Functional Exit Equivalence (FEE)¹: if one node is captured, system variety is preserved through other federated nodes.
- **Mobile Edge (Local Inference):** Transitioning to local execution on mobile devices (using Small Language Models) provides the strongest EXIT guarantee. Users perform essential functions without transmitting telemetry to central authority, bypassing Rule of Workflow capture.

Principle 8.1 (Edge Compute Capture Reduction). Decentralization reduces Compute Capture risk: the cost of running infrastructure is distributed across the community’s own hardware, lowering the resource invariance barrier κ .

8.1.2 CODE as Bytecode: Verification vs. Inspection

Bytecode and **Reproducible Builds** are the technical prerequisites for CODE in modern DPI:

- **Logic Verification:** Transparency is hollow if published source code does not match the binary running on edge devices. The framework requires Reproducible Builds so any third party can compile source into identical bytecode, verifying that “Logic” matches “Affidavit.”
- **Wasm/Bytecode Portability:** Standardized, portable bytecode formats (like WebAssembly) avoid vendor lock-in. This lowers WCC because orchestration logic can be ported between cloud or edge environments, satisfying FORK.

Architectural Implication. By adopting Edge/Bytecode topologies, states move from **Rule of the Ledger** to **Rule of the Protocol**, ensuring that administrative power remains corrigible even as it scales toward universal necessity. The substrate shifts from centralized cloud to distributed edge, but the five tests remain invariant.

¹FEE is defined and formalized in the companion paper [Aravind, 2026]. When literal exit is impossible for essential services, FEE provides architectural guarantees that recreate the error-signal strength of market exit without requiring citizens to abandon society.

Table 9: Corrigibility impact of Edge/Bytecode topologies

Test	Edge/Bytecode Impact	Cybernetic Function
EXIT	High (local refusal possible)	Error Signal
CODE	Verified (via reproducible bytecode)	Transmission Clarity
AUDIT	Fragmented (requires federated telemetry)	Sensor
GOVERN	Challenging (requires federated guardrails)	Actuator
FORK	Excellent (low compute multiplier κ)	Selection Pressure

9 Anticipated Objections

Before discussing implications, we address two fundamental objections that challenge the framework’s applicability to learned systems.

9.1 Objection: Deterministic Tests Cannot Apply to Stochastic Systems

A natural critique holds that applying rigid, deterministic structural tests to systems that are inherently stochastic constitutes a category error. How can tests designed for inspectable code apply to systems where identical inputs yield probabilistic outputs and “logic” is encoded in billions of weights?

Response: Verification Methods Evolve, Standards Remain. The framework acknowledges that learned systems dismantle the assumption that source code is the sole arbiter of behavior. However, this does not invalidate structural corrigibility. It *strengthens* its necessity. As systems become less intelligible, external correction mechanisms become more critical, not less.

The key insight is that the **standard remains invariant** while the **verification method adapts**:

- **CODE** shifts from reading source code to requiring the **LWD-R Triad**: Logic (architecture), Weights (parameters), Data (provenance), and Representation (ontological categories).
- **AUDIT** shifts from discrete logic inspection to statistical bounds and continuous monitoring.
- **FORK** shifts from file copying to resource forkability, measured by access to compute and training data, not just open weights.
- **GOVERN** requires deterministic guardrails *around* the stochastic model, governing objectives and value tradeoffs at the infrastructure layer.

Deterministic Boundaries as Engineering Requirement. The framework argues that deterministic structural boundaries are not incompatible with AI but are an *engineering requirement* for the agentic economy. Autonomous agents lack biological resilience for ambiguity. To an AI agent, the five tests function as a necessary API contract:

- EXIT = Exception handling (preventing logic deadlocks)
- CODE = Documentation (enabling valid plan formation)
- AUDIT = Observability (providing reliable failure signals)
- GOVERN = Constraint discovery (since “advisory” guidelines produce undefined behavior)

The Cost of Abandoning Structural Evaluation. If structural evaluation is set aside on the grounds that “AI is too stochastic to be evaluated by structural rules,” governance defers to operators by default: the only remaining standards are those operators choose to publish. Structural evaluation methods are therefore necessary, not optional. The framework does not attempt to inspect the probabilistic “thoughts” of AI models. Instead, it demands structural control over the *pipeline that builds them* (data, compute, representation) and the *infrastructure that deploys them* (workflow orchestrations and guardrails).

9.2 Objection: Compute Capture Makes FORK Trivially Impossible

If frontier model training costs exceed global academic compute capacity by orders of magnitude, doesn’t the framework simply conclude “all frontier AI fails FORK,” a trivially true and useless finding?

Response: The Distinction Is Training vs. Inference Forkability. The framework distinguishes *inference forkability* (running weights) from *training forkability* (reproducing the model). Only training

forkability satisfies FORK for governance purposes. The evidence requirements in Table 6 are achievable: published training code, data manifests, and compute cost estimates.

The claim is not that all AI fails FORK, but that systems *withholding these artifacts* fail. OLMo, Pythia, and academic models demonstrate that FORK-compliant AI is possible. The question is whether frontier labs *choose* to comply.

The framework implies that **compute is itself DPI**. Legal forkability without practical compute accessibility is hollow. Public compute infrastructure becomes a governance requirement, not a reason to abandon the standard.

10 Epistemic Constraint Infrastructure

Corrigibility ensures that systemic error can be contested and corrected. It does not guarantee epistemic rigor within system operation. In verifiable domains such as software development, symbolic constraint systems (compilers, type systems, unit tests) provide hard pass/fail validation of outputs. These mechanisms enforce epistemic discipline through execution.

In learned or research-driven domains, comparable constraint infrastructure is often absent. Validation loss improvements, heuristic metrics, or informal review processes may not provide sufficient structural enforcement to prevent spurious inference.

Corrigible epistemic infrastructure therefore requires not only transparency and forkability, but engineered constraint mechanisms that reject inadequately controlled claims. Without such constraint, systems may remain structurally corrigible yet epistemically unstable.

10.1 Illustrative Example: Multi-Agent Research Organization

Consider a multi-agent research organization attempting to optimize a model parameter. The organization may satisfy structural corrigibility conditions: independent branches (FORK), transparent code (CODE), internal review (AUDIT), and supervisory governance (GOVERN). Yet if experimental protocols do not enforce baseline normalization, compute accounting, or ablation requirements, agents may report spurious improvements.

The system remains corrigible (errors can be contested), but epistemic progress is not guaranteed. This distinction clarifies the boundary between structural legitimacy and scientific competence.

Principle 10.1 (Corrigibility vs. Competence). Structural corrigibility ensures that claims can be challenged and systems can be corrected. Epistemic quality requires additional constraint mechanisms that function as “compilers for reality,” rejecting outputs that fail verification.

11 Discussion

11.1 Implications for AI Governance

This framework offers three contributions to AI governance discourse:

1. **Structural vs. Behavioral Focus.** Most AI governance frameworks focus on behavioral outcomes (bias, accuracy, harm). This framework focuses on structural preconditions: can affected parties correct the system regardless of what harms emerge? Structural corrigibility is necessary for any behavioral governance to function.
2. **Distinguishing Open-washing.** The LWD-R requirement and compute capture analysis provide falsifiable tests for “open AI” claims. Systems publishing weights without training data, code, or accessible compute achieve only inference forkability: performative transparency without correction capacity.
3. **Agentic Economy Readiness.** As autonomous agents proliferate, infrastructure must become machine-readable. The five tests, reinterpreted as API contracts, specify the minimum requirements for agentic interoperability. Incurable infrastructure blocks the agentic economy.

11.2 Limitations

1. **Threshold calibration is underdetermined.** The compute capture threshold κ , WCC trigger values, and RCR operationalization require empirical calibration that this paper does not provide.
2. **Rapid technological change.** AI architectures evolve faster than governance frameworks. The risk profiles in Table 8 may become obsolete as new architectures emerge.
3. **LWD-R is aspirational.** No current frontier model satisfies full LWD-R disclosure. The requirement describes what *should* exist for corrigibility, not what currently does.
4. **Compute accessibility is geopolitically constrained.** Public compute infrastructure sufficient for training forkability may be infeasible for many nations due to cost, expertise, and supply chain constraints.
5. **The framework does not address emergent capabilities.** Unpredictable capabilities that emerge during training present governance challenges that structural tests alone cannot address.
6. **Deterministic boundaries create known attack surfaces (SBOM problem).** The Action Boundary Protocol requires explicit, machine-readable disclosure of action specifications, permitted tools, and execution constraints. This transparency enables governance but simultaneously creates a complete map of the system’s action space for adversaries. The same allowlist that prevents unauthorized actions also reveals exactly which actions *are* authorized. This is analogous to Software Bill of Materials (SBOM) disclosure: useful for auditing dependencies but also useful for targeting known vulnerabilities. All deterministic governance systems face this tradeoff. The boundaries that constrain must be known, and what is known can be attacked. The framework does not resolve this tension; it makes it explicit.

11.3 Future Work

1. **Empirical RCR measurement:** Developing quantitative methods to assess ontological contestability
2. **Compute accessibility indices:** Cross-national comparison of training forkability capacity
3. **GDoS simulation:** Modeling governance failure under agentic scaling
4. **LWD-R schema standardization:** Machine-readable formats for disclosure requirements
5. **Regulatory integration:** Mapping framework tests to emerging AI regulations (EU AI Act, etc.)
6. **Centralized vs edge architecture:** Analyzing corrigibility trade-offs between centralized compute (more auditable, more capturable) and edge deployment (capture-resistant, audit-complex)

12 Conclusion

The five corrigibility tests remain invariant for learned systems. What changes is the verification method and the resource barriers to compliance. This paper extends the framework through: LWD-R disclosure requirements extending CODE to training pipelines, Compute Capture as a resource barrier that transforms “open weights” into performative transparency, and the Action Boundary Protocol separating stochastic inference from deterministic execution.

Three implications:

1. **For policymakers:** The framework provides falsifiable tests for AI infrastructure procurement. Systems failing LWD-R or lacking training forkability should trigger governance scrutiny regardless of “open” marketing claims.
2. **For technologists:** The Action Boundary Protocol offers a practical architecture for building corrigible AI systems. Deterministic validators wrapping stochastic inference satisfy GOVERN without requiring interpretability of model internals; multiple open protocols can instantiate the requirement.
3. **For citizens:** Infrastructure that affects populations at scale must be structurally correctable by those it governs, regardless of whether it executes deterministic code or learned parameters.

Liability and Enforcement. Architectural corrigibility must be complemented by clear liability and enforcement semantics: for any failure mode, supply-chain roles (principal, integrator, model provider, operator, trustee, as distinguished in EU AI Act Article 25 et seq.) must be mappable to legal responsibility and to mandated mitigation steps, so that evidence produces enforceable consequences rather than forensic narrative alone.

Adversarial Robustness. This framework assumes good-faith actors operating within institutional constraints. It does not address adversarial manipulation: coordinated gaming of audit mechanisms, governance capture through procedural compliance, prompt injection attacks on action boundaries, or fork-and-poison attacks on model provenance. Extension to adversarial settings requires additional machinery beyond this paper’s scope.

Core Thesis. The corrigibility invariant extends to stochastic systems: systems remain correctable if and only if the five conditions close the feedback loop under probabilistic verification. Learned systems do not escape the requirement; they raise the verification burden. Corrigibility is not a deployment certification. It is a persistence condition.

Acknowledgments

This paper extends the DPI corrigibility framework [Aravind, 2026] to learned systems. The author thanks the AI governance research community for ongoing critique.

License

This work is released under CC0 1.0 Universal (Public Domain).

Data and Code Availability

The complete framework is available across two repositories:

- **Paper Repository:** <https://github.com/anivar/corrigibility-framework>
 - Theoretical framework and proofs
 - Case studies and empirical analysis
- **Schema Repository:** <https://github.com/anivar/corrigibility-schema>
 - **DPI Schemas:** `schema/dpi/infrastructure.json`, `schema/dpi/audit.json`
 - **EPI Schemas:** `schema/epi/infrastructure.json`, `schema/epi/audit.json`
 - **Assessment Rules:** `rules/assess/` — Test definitions including LWD-R transparency criteria, Compute Capture detection, Action Boundary verification, and Variety Drift monitoring
 - **Protocol:** Versioning, canonicalization, signature standards
 - **Examples:** Fictional reference implementations

The schema repository encodes the architectural requirements of Section 6.6 in declarative JSON; it is intentionally agnostic to any specific action-specification standard and is versioned independently from the theoretical papers to allow schema evolution as DPI/EPI systems mature.

The schema architecture demonstrates that the five tests remain invariant across DPI and EPI. Only the verification method and evidence requirements adapt to the system type. The separation into creator, evaluator, and assessment components enables independent tooling for operators, auditors, and validators.

A Action Boundary Schema Specifications

To distinguish valid governance from performative compliance, the framework provides machine-readable assessment schemas for agentic workflows. These schemas convert the theoretical defense against Workflow Capture into enforceable compliance gates. The fields are protocol-agnostic: any open action-specification format that satisfies the five architectural requirements of Section 6.6 can populate them.

A.1 Operator’s Affidavit: `infrastructure.json`

The operator must declare exactly how the stochastic model is bounded. The `agentic_workflow_governance` object proves the system is using deterministic, externally specified action definitions rather than proprietary black-box orchestration.

```
{
  "$schema": "https://github.com/anivar/corrigibility-schema/blob/main/schema/epi/infrastructure.json",
  "system_name": "State Welfare Triage Agent",
  "agentic_workflow_governance": {
    "workflow_type": "bounded_action_orchestration",
    "action_specification_format": "open-declarative-schema",
    "action_specification_url": "https://github.com/state-gov/welfare-actions",
    "execution_environment_isolation": {
      "pre_execution_validation": true,
      "post_execution_validation": true,
      "constraints_outside_model_context": true
    },
    "independent_audit_path": {
      "logs_publicly_signed": true,
      "validator_source_disclosed": true
    },
    "epistemic_delegation_metrics": {
      "ontology_substitutability": 0.9,
      "model_agnostic": true,
      "vendor_lock_in_risk": "low"
    }
  }
}
```

Critical Fields.

- **action_specification_format**: Names the open, declarative schema in which actions, preconditions, and termination criteria are described. The schema must be independently retrievable and parseable; any standard satisfying the five architectural requirements is admissible.
- **constraints_outside_model_context**: The cybernetic anchor. Legally binds the operator to the claim that governance constraints are hard-coded in the validator’s execution environment, not merely injected into the model’s context window (which can be bypassed via prompt injection).
- **independent_audit_path**: Asserts that an external auditor can replay validator decisions against signed logs without operator cooperation.
- **model_agnostic**: Asserts high model portability ($M_{\text{portability}}$): the state can swap the underlying model without rewriting the policy workflow.

A.2 Auditor’s Finding: `audit.json`

The auditor does not merely read the operator’s JSON; they red-team it. The auditor attempts to bypass action specifications and calculates the actual Workflow Capture Coefficient (WCC).

```
{
  "$schema": "https://github.com/anivar/corrigibility-schema/blob/main/schema/epi/audit.json",
  "target_system": "State Welfare Triage Agent",
  "workflow_capture_assessment": {
    "wcc_score": 0.15,
    "wcc_threshold_passed": true,
    "action_boundary_tests": {
      "validator_bypassed_via_prompt_injection": false,
      "unauthorized_action_invocation_possible": false,
      "termination_criteria_verifiable": true
    },
    "reproduction_tests": {
      "actions_reproducible_with_alt_model": true,
      "ontology_schema_publicly_available": true
    }
  },
  "determinations": {
```

```

    "CODE": true,
    "GOVERN": true,
    "FORK": true
  }
}

```

A.3 Enforcement Logic Gates

The schema fields directly map to corrigibility test determinations:

1. **Enforcing GOVERN:** If `validator_bypassed_via_prompt_injection` is `true`, the system instantly fails GOVERN. The constraints were probabilistic (advisory to the model) rather than deterministic (binding).
2. **Enforcing CODE (LWD-R):** If `ontology_schema_publicly_available` is `false`, the system fails the Representation (R) requirement of CODE, triggering Representation Capture Risk (RCR).
3. **Enforcing FORK:** If `actions_reproducible_with_alt_model` is `false`, the workflow is tightly coupled to a proprietary vendor. The WCC score spikes, and the system fails FORK.

Table 10: Schema field to corrigibility test mapping

Schema Field	Test	Failure Condition
<code>constraints_outside_model_context</code>	GOVERN	<code>false</code> $\Rightarrow$ constraints are advisory
<code>validator_bypassed_via_prompt_injection</code>	GOVERN	<code>true</code> $\Rightarrow$ boundaries are soft
<code>ontology_schema_publicly_available</code>	CODE	<code>false</code> $\Rightarrow$ RCR triggered
<code>actions_reproducible_with_alt_model</code>	FORK	<code>false</code> $\Rightarrow$ vendor lock-in
<code>wcc_score</code>	FORK	<code>> 0.7</code> $\Rightarrow$ high capture risk

Architectural Significance. By encoding the Action Boundary requirements into adversarial JSON schemas, the framework transforms theoretical governance requirements into machine-readable, enforceable compliance gates. State architects can require these schemas in public procurement contracts, definitively banning black-box, vendor-locked agentic workflows. The schema is intentionally agnostic to any specific action-specification standard; conformance is judged structurally against the five requirements, not by branding.

B Glossary of Key Terms

The glossary below is shared verbatim with the companion paper [Aravind, 2026] so that terminology is identical across the pair.

Action Boundary

The deterministic envelope that wraps stochastic inference in an agent system. Inference outputs (action proposals) must pass through a hard-coded validation layer before execution. The action boundary is the architectural locus of GOVERN in agentic systems.

Action Boundary Protocol

The set of five architectural requirements (deterministic validation, context-window isolation, exception handling, machine-readable specifications, binding constraints) that an agent system’s action boundary must satisfy to support corrigibility. Independent of any specific external standard.

AUDIT

Test 3 of corrigibility. Independent verifiability: third parties can verify that the system’s behavior matches its disclosed specification through tamper-evident logs, statistical bounds, or drift monitoring.

CODE

Test 2 of corrigibility. Inspectability of the system’s execution. For deterministic DPI, this is source-code disclosure; for learned systems (EPI) it is the LWD-R requirement.

Compute Capture

A FORK failure mode specific to learned systems. Occurs when the compute cost to retrain a functionally

equivalent model (C_{train}) exceeds the compute accessible to non-operator actors ($C_{\text{accessible}}$) by more than a policy-determined multiplier κ .

Corrigibility

The structural capacity of those affected by a system to detect error, signal harm, and trigger correction without incurring material loss or irreversible consequence. Not a moral preference but an architectural stability requirement.

Data Capture

A FORK failure mode parallel to Compute Capture. Occurs when the data required to train a functionally equivalent model (D_{required}) exceeds the data accessible to non-operator actors ($D_{\text{accessible}}$) by more than a policy-determined multiplier κ_D . Encompasses synthetic data, distilled outputs, RLHF traces, reward models, and proprietary evaluation sets.

Digital Public Infrastructure (DPI)

Shared digital systems — ledgers, registries, payment rails, data exchanges — intended to deliver public services at population scale. The subject of the deterministic-systems paper in this pair.

Epistemic Public Infrastructure (EPI)

Public infrastructure whose behavior is generated by learned parameters rather than explicit logic. Includes AI-based identity, eligibility, triage, and orchestration systems. The subject of the learned-systems paper in this pair.

EXIT

Test 1 of corrigibility. Reversibility of participation: affected parties can refuse or leave the system without prohibitive penalty, or are guaranteed Functional Exit Equivalence when literal exit is infeasible.

FORK

Test 5 of corrigibility. Independent reproduction: the system can be recreated by parties outside operator control using only public artifacts.

Functional Exit Equivalence (FEE)

Architectural guarantees that recreate the error-signal strength of literal exit when literal exit is impossible (e.g., for monopoly identity systems). FEE provides federation, parallel operation, and revocable participation as substitutes for market exit.

Governance Denial of Service (GDoS)

Failure mode in agentic systems where action throughput outruns governance correction velocity. Without explicit termination criteria and bounded action specifications, agents loop indefinitely or escalate beyond legitimate authority.

GOVERN

Test 4 of corrigibility. Constitutive constraint: affected populations have binding authority over the system, not merely advisory input. For deterministic DPI this is RFC/charter governance; for EPI it is the action boundary protocol.

LWD-R

Four-layer transparency requirement for learned systems: **L**ogic (architecture and inference code), **W**eights (trained parameters), **D**ata (training corpus with provenance), and **R**epresentation (the categorical schema the model uses to interpret inputs).

Operative Representation

The categorical schema that emerges during inference under specific deployment conditions (quantization, MoE routing, output filtering), as distinguished from *nominal representation* (the latent space of the trained artifact in isolation). Disclosure must describe operative R, not nominal R.

Provenance Chain

The sequence $P = [g_0, g_1, \dots, g_n]$ of generators producing training data for downstream models. Transparency does not survive an opaque link in this chain.

Representation Capture Risk (RCR)

The risk that a learned system’s representational categories (“eligibility,” “risk,” “fraud”) become uncontrollable administrative facts. Operates upstream of the action boundary.

Requisite Variety

Ashby’s Law: a controller must possess at least as many states as the system it regulates. Applied to DPI/EPI: governance mechanisms must match the population’s diversity of conditions to remain corrective.

Rule of the Ledger / Rule of the Workflow

Successive regimes in which authority is exerted through synchronized, self-executing artifacts. The Rule of the Ledger describes deterministic record-based authority; the Rule of the Workflow describes orchestration-mediated authority in agentic infrastructure.

Variety Drift / Temporal Variety Gap

The growing gap between a frozen model’s variety and the evolving environmental variety it is meant to regulate. Drives corrigibility from a one-shot certification into a persistence condition.

Workflow Capture Coefficient (WCC)

A diagnostic abstraction quantifying epistemic delegation risk as a function of reproduction cost, ontological substitutability, and model portability. WCC near 1 indicates near-total capture; WCC near 0 indicates high sovereignty. Computed as a harmonic mean to ensure weakness in any dimension cannot be hidden by strength in others.

References

- Daniel Abadie. DPI-AI framework: Building AI-ready nations through digital public infrastructure. Centre for Digital Public Infrastructure (CDPI), 2026. <https://digitalpublicinfrastructure.ai>.
- Anivar A. Aravind. Corrigibility as a structural precondition for digital public infrastructure: A cybernetic framework. <https://github.com/anivar/corrigibility-framework>, 2026. Establishes the five-test framework (EXIT, CODE, AUDIT, GOVERN, FORK) for deterministic DPI.
- W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, London, 1956.
- European Parliament and Council of the European Union. Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, 2024.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- Dylan Hadfield-Menell, Anca Dragan, Pieter Abbeel, and Stuart Russell. The off-switch game. In *Proceedings of the AAAI Workshop on AI, Ethics, and Society*, 2017.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.
- Open Source Initiative. The open source ai definition 1.0. <https://opensource.org/ai/open-source-ai-definition>, 2024.
- Ilya Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget. *arXiv preprint arXiv:2305.17493*, 2023. doi: 10.48550/arXiv.2305.17493.
- Nate Soares and Benja Fallenstein. Agent foundations for aligning machine intelligence with human interests: A technical research agenda. Technical Report Technical Report 2014–8, Machine Intelligence Research Institute, 2017.